

PRACTICAL USER MANUAL:

Upscale + Frame generator (FSR + IFS)

HARDWARE REQUIREMENTS AND DEPENDENCIES

In order for the post-processing pipeline to work properly on your computer, your system must comply with the following physical and software dependencies:

- **Vulkan API support (REQUIRED):** Your graphics card must natively support Vulkan technology. The IFS authorial shader runs the time calculations directly on the GPU silicon via this API. It works on dedicated graphics cards (AMD RX, Nvidia GTX/RTX) and also on integrated chips (AMD Vega or newer Intel HD Graphics). If your GPU does not support Vulkan, the process will not start.
- **CPU:** Any basic processor with 4 or 6 old cores (such as the AMD FX or Intel Core line of past generations) is perfectly capable of running the pipeline. Average CPU usage is around 45%, keeping Windows completely free of crashes or overheating during rendering.
- **Video Memory (VRAM):** Legacy video cards with 2GB of VRAM can do the job smoothly if the user operates smartly. It is recommended to close internet browsers, Discord, and background games before starting the process at high resolutions (such as 2K or 4K) to avoid overflowing the card's memory.
- **Operating System:** Windows 10 or Windows 11 64-bit with the PowerShell terminal active and released for script execution.

INSTALLATION

Start by downloading the zip with the contents. Unzip wherever desired. You'll see the entire FSRIFS folder structure.

The whole process unfolds within a powershell prompt (knowing basic commands for navigating folders via command is important).

Let's start by opening the powershell, it doesn't need to be in adm mode, the script won't need privileges to run.

If this is your first time running scripts on your computer, enter the command below to allow scripts to run:

```
PowerShell
```

```
# Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process
```

Answer YES to apply the enforcement policy.

Ok! Now we're ready to run the processes and you can do it in two ways:

- Dragging and Dropping your video file onto an automated bot that performs the entire process for you (aimed at users without prompt knowledge)
- Prompt running the scripts to run the pipeline (prompt knowledge required)

Drag & Drop

This is the easiest way to run the pipeline. Using the file explorer, go to your folder where you unzipped FSRIFS, you will see a file called "DRAG_VIDEO_HERE.bat", now open another file explorer window and go to the folder where you have the video you want to process. Drag your video and drop it over the file we mentioned.

You can configure the process by editing the file: "DRAG_CONFIG.txt":

```
[DRAG_SETTINGS]
SCALE=1920x1080
SHARPNESS=5
FPS=60
QUALITY=med
VERBOSE=false
```

Powershell

This is the original method for you to process your videos. We need to access the location where you unzipped the FSRIFS, in my case the location looks like this:
D:\FSRIFS

PowerShell

```
# cd "D:\FSRIFS"
```

You will then be in the pipeline folder and you can run two scripts:

process.ps1:

(our main script with it we can upscale and or generate fps)

The script acts in a modular way, you can access these modules through parameters on the command line, let's get to know them:

- **-file "nome_do_arquivo.mp4"** The full path of the video file you want to process. It should be enclosed in quotation marks (e.g., "C:\folder\video.mp4").
- **-scale factor|resolution** Final dimension of the video. If omitted, the upscale does not happen and the original size is maintained. The amount can be informed in the following formats:

- multiplier factor for the resolution gain. Use decimal numbers (e.g., the dimension factor of a video from 720p to 1080p is 1.5)
 - literal resolution where you can inform for example: "1920x1080"
- **-fps number:** The desired frame rate for fluidity (e.g. 60). If omitted, frame generation does not happen and the original FPS is retained.
- **-quality "low|med|big"** The weight, compression and size profile of the final file. If you omit this parameter, the system automatically assumes the balanced profile "med". The available options are:
 - low: Focus on drastic space savings. Reduces the weight of the final file and alleviates the use of VRAM. Generates a small final file.
 - med: The ideal balance point. Maintains edge sharpness and visual fidelity without blowing out the HD. Generates a final file of medium size.
 - big: Maximum visual fidelity and high bitrate, aimed at archiving or uploading in high quality. Generates a huge final file.
- **-Sharpness number** Quality applied by FSR going from 0 (minimum) up to 10 (maximum).
- **-v** Verbose mode, where detailed information about the process is listed in the terminal.

Security Note: The script requires you to use at least -scale or -fps.

Examples:

- **Upscale ONLY: let's say from 720p to 1080p, with average sharpness, then we apply a 1.5x scale and sharpness 5**

PowerShell

```
.\process.ps1 -file "C:\folder\video.mp4" -scale 1.5 -sharpness 5
```

- **Do Frame Generation ONLY: Let's say your original video has 30fps**

PowerShell

```
.\process.ps1 -file "C:\folder\video.mp4" -60 fps
```

- **Full Pipeline (Upscale 1080p + 60 FPS frame generation + checking a detailed log at the prompt)**

PowerShell

```
.\process.ps1 -file "C:\folder\video.mp4" -scale "1920x1080" -fps 60 -v
```

The script will identify the source of your file and saves the final result **in the exact same folder as the original video**. The name of the new file is intelligently generated with suffixes based on what you choose to do:

- If you only used upscale: video_FSR_1080x720.mp4
- If you only used the frames: video_60fps.mp4
- If you used the full pipeline: video_FSR_1080x720_60fps.mp4

extract.ps1:

(script to extract a few frames from the processed video so you can evaluate via frame if the result is good)

You can extract all x second frames starting at a certain time from the video. It is the ideal tool for you to visually evaluate the quality of the final result, allowing you to check frame by frame.

Command-line parameters:

- **-file "file":** The full path of the video file you want to process. It should be enclosed in quotation marks (e.g., "C:\folder\video.mp4").
- **-time "hh:mm:ss":** The starting point of the extraction in the format HH:MM:SS. If it is not filled, it will automatically start at 3 seconds (00:00:03).
- **-secs number:** How many seconds of video you want to turn into photos. If it is not filled, it will only extract 1 second (if the video is at 60fps, then 60 frames will be extracted).
- **-output "destination folder":** Enclosed in double quotation marks. Folder where to save the photos. If not populated, the script will extract the frames into the output folder at the root of the FSRIFS.

KNOWN ISSUES / TECHNICAL LIMITATIONS

- **Blur or Smoothing on Pause (Freeze Frame Artifacts):**
 - **Symptom:** When pausing the final video during fast-moving scenes, the image may appear soft, blurry, or have a "ghosting" effect.
 - **Cause:** The video player froze the playback clock exactly on top of an intermediate (interpolated) frame generated by the temporal mathematics of the IFS shader

- **Solution:** There is nothing to fix. This behavior is the mechanics needed to ensure the illusion of fluidity in frame generation with video in motion, without requiring heavy AI or hardware with Tensor Cores. It is the price you pay for a solution aimed at lowend pc.

CREDITS AND TECHNICAL SUPPORT

- **Ideation, Authored** by Shader IFS (MIT) and Pipeline Integration: by Aless (MaulSmoke).

- **Third Party Technologies Used:** FFmpeg (GPL), libplacebo (LGPL), AMD FidelityFX FSR (MIT)

- **Official Project Channel (YouTube):** Watch tutorials, news, performance showcase and ask your questions directly in the channel's community:
<https://youtube.com/@toplayaless?si=OjRmKWfSgl5tDydC>